MULTI-HAZARD ROAD DAMAGE DETECTION AND UNIFIED RISK-BASED MAINTENANCE PRIORITIZATION USING DEEP LEARNING

A PROJECT REPORT

Submitted by

A. HEMA HARSHITHA - CH.SC.U4AIE24005
K. CHANDANA - CH.SC.U4AIE24035
K. NAGA HASMITHA - CH.SC.U4AIE24037

in partial fulfillment for the award of the degree of
BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING

Under the guidance of
Dr.M.PALLAVI
Department of Computer Science & Engineering



AMRITA VISHWA VIDYAPEETHAM
AMRITA SCHOOL OF COMPUTING
CHENNAI – 601103
October 2026

BONAFIDE CERTIFICATE

Certified that this project report entitled "MULTI-HAZARD ROAD DAMAGE DETECTION AND UNIFIED RISK-BASED MAINTENANCE PRIORITIZATION USING DEEPLEARNING"is the bonafide work of "A.HEMAHARSHITHA (CH.SC.U4AIE24005), K. CHANDANA (CH.SC.U4AIE24035) & K. NAGA HASMITHA (CH.SC.U4AIE24037) who carried out the project work under my supervision.

SIGNATURE

Dr. S. BAGHAVATHI PRIYA
CHAIRPERSON
Department of CSE
Amrita School of Computing
Chennai – 601103.

SIGNATURE

Dr.M.Pallavi
SUPERVISOR
Dept. of CSE
Amrita School of Computing
Chennai – 601103.

INTERNAL EXAMINER

EXTERNAL EXAMINER



DECLARATION BY THE CANDIDATES

We declare that the report entitled "MULTI-HAZARD ROAD DAMAGE DETECTION AND UNIFIED RISK-BASED MAINTENANCE PRIORITIZATION USING DEEP LEARNING" submitted by us for the degree of Bachelor of Technology in Computer Science and Engineering is the record of the project work carried out by us under the guidance of "Dr.M.Pallavi" and this work has not formed the basis for the award of any degree, diploma, associateship, fellowship, titled in this or any other University or other similar institution of higher learning.

SIGNATURE OF THE CANDIDATES

A. HEMA HARSHITHA (CH.SC.U4AIE24005)
K. CHANDANA (CH.SC.U4AIE24035)
K. NAGA HASMITHA (CH.SC.U4AIE24037)

Department of Computer Science & Engineering (AIE)
Amrita School of Computing, Chennai Campus.

ABSTRACT

Road damage is usually found by manual inspection, which is slow to repeat and leaves the findings disconnected from later maintenance decisions. A detector can locate defects in a photograph, but it does not by itself keep the evidence, record where the photograph was taken, or give a consistent basis for deciding which record to review first.

This project builds RoadGuard AI, an inspection application around a YOLOv8s detector for four classes: longitudinal cracks (D00), transverse cracks (D10), alligator cracks (D20) and potholes (D40). The detector was trained on a prepared RDD2022 collection of 35,490 images, split into 24,976 training, 5,339 validation and 5,175 held-out test images. The checkpoint is exported to ONNX and runs in the browser, so the default workflow needs no separate Python server. A rule-based screening priority S = min(100, round(B + 20A + K + C)) combines the highest class baseline, box coverage, detection count and context inputs, and leaves prediction confidence out on purpose. Records keep original and annotated images, location where available and review status, and can be shown on a map and exported as PDF, CSV and JSON.

On the held-out test split the model reached precision 67.71%, recall 52.83%, mAP50 59.02% and mAP50–95 30.51%. D20 had the strongest AP50 (68.25%) and D40 the weakest (49.57%), with pothole recall of 43.16%, so missed potholes are a significant limitation. A Tesla T4 benchmark averaged 15.50 ms per image. Recorded software checks passed, including checkpoint checksum, ONNX parity and 20 backend tests. Moving-vehicle performance, physical-device camera and GPS behaviour, offline reopening and browser PDF layout are not verified, the priority rule is not validated against engineer-assigned severity, and the test split is not route-disjoint.

Keywords: Road Damage Detection, YOLOv8, RDD2022, Maintenance Prioritization, ONNX Runtime Web, Pothole Detection, Inspection Records, mAP, Screening Priority, Offline Web Application.

ACKNOWLEDGEMENT

This project work would not have been possible without the contribution of many people. It gives me immense pleasure to express my profound gratitude to our honorable Chancellor Sri Mata Amritanandamayi Devi, for her blessings and for being a source of inspiration. I am indebted to extend my gratitude to our Administrative Director, Swami Vinayamritananda Puri and Campus Director, Mr. I B Manikantan, Amrita School of Computing and Engineering, for facilitating us all the facilities and extended support to gain valuable education and learning experience.

I register my special thanks to Dr. V. Jayakumar, Principal, Amrita School of Computing and Engineering for the support given to me in the successful conduct of this project. I wish to express my sincere gratitude to Dr. S. Baghavathi Priya, Chairperson, Dr. Annamalai R, Program Chair, and to my supervisor Dr.M.Pallavi , Associate Professor, Department of Computer Science and Engineering, for their inspiring guidance, personal involvement, and constant encouragement during the entire course of this work.

I am grateful to Project Coordinator, Review Panel Members, and the entire faculty of the Department of Computer Science & Engineering, for their constructive criticisms and valuable suggestions which have been a rich source to improve the quality of this work.

A. HEMA HARSHITHA (CH.SC.U4AIE24005)
K. CHANDANA (CH.SC.U4AIE24035)
K. NAGA HASMITHA (CH.SC.U4AIE24037)
Department of Computer Science & Engineering (AIE)
Amrita School of Computing, Chennai Campus.

TABLE OF CONTENTS

TITLE	PAGE NO.
Abstract	**iv**
List of Figures	**viii**
List of Tables	**ix**
List of Symbols and Abbreviations	**x**
CHAPTER 1 INTRODUCTION	**1**
1.1 Background and Evolution of Road Damage Detection	1
1.2 Problem Statement	2
1.3 Motivation for Unified Risk-Based Maintenance Prioritisation	3
1.4 Objectives of the Project	3
1.5 Scope and Operational Boundaries	4
1.6 Organization of the Report	4
CHAPTER 2 LITERATURE REVIEW	**6**
2.1 Road Damage Datasets and Benchmarks	6
2.2 Object Detection Architectures and Training Objectives	7
2.3 Smartphone Sensing, Deployment and Mapping	7
2.4 Data Preparation, Leakage and Evaluation	8
2.5 Condition Assessment and Maintenance Prioritisation	8
2.6 Critical Synthesis and Identified Research Gaps	8
CHAPTER 3 MATHEMATICAL MODELING & THEORETICAL FORMULATIONS	**11**
3.1 Image Model and Detector Output	11
3.2 Letterboxing and Output Decoding	11
3.3 Training Objective	12
3.4 Overlap, Confidence Filtering and Non-Maximum Suppression	12
3.5 Evaluation Metrics	13
3.6 Unified Screening Priority Formulation	14
3.7 Location, Retry and Revision Rules	15
CHAPTER 4 SYSTEM ARCHITECTURE AND IMPLEMENTATION	**17**
4.1 Platform Architecture Overview	17

4.2 Datasets and Data Pipeline	19
4.3 Detector Implementation and Checkpoint Integration	21
4.4 Browser Inference and Priority Implementation	23
4.5 Camera, Location and Map Implementation	24
4.6 Web Application, Storage and Reporting	25
4.7 Optional Backend, Software Environment and Reproducibility	28
CHAPTER 5 RESULTS, BENCHMARKS, AND DISCUSSION	**31**
5.1 Experimental Setup and Evaluation Protocol	31
5.2 Training Behaviour from Saved Logs	31
5.3 Held-Out Test Performance	33
5.4 Class-Wise Performance	33
5.5 Precision–Recall Behaviour and Confidence Threshold	35
5.6 Comparison with the YOLO11s Notebook Experiments	36
5.7 Prediction and Priority Worked Examples	37
5.8 Software Validation and Verification Status	39
5.9 Discussion	41
CHAPTER 6 CONCLUSION AND FUTURE SCOPE	**43**
6.1 Summary of Completed Contributions	43
6.2 Key Theoretical and Practical Takeaways	43
6.3 Limitations	44
6.4 Future Research Directions	44
REFERENCES	**45**

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Figure 3.1	Letterbox Preprocessing and Coordinate Restoration	12
Figure 3.2	Screening Priority Calculation. Four Terms Are Added, Rounded, Capped at 100 and Mapped to a Band; Confidence Does Not Enter	15
Figure 4.1	System Architecture. Explanatory Implementation Pipeline from Photograph to Saved Record; Review Can Occur after Initial Saving	18
Figure 4.2	Dataset Composition. Left: Images per Partition. Right: Instances per Class in the Prepared Dataset	19
Figure 4.3	Sample RDD2022 Ground-Truth Annotations (Red Boxes) for the Four Retained Target Classes; Rows Show Original Class IDs 0, 1, 2 and 4	20
Figure 4.4	YOLOv8s Detector Structure and the Application's Post-Processing (Schematic)	22
Figure 4.5	Historical Website Screenshot Taken before the Optional-Location and Model-Loader Fixes (Loading State; Not Proof of Current Successful Detection)	26
Figure 5.1	Recorded Training and Validation Box, Classification and Distribution Focal Losses	32
Figure 5.2	Validation mAP across Logged Epochs; the Dashed Line Marks the Highest Logged mAP50–95 (Epoch 87)	32
Figure 5.3	Class-Wise Held-Out AP50 and Recall from test_metrics.json	34
Figure 5.4	Saved Deployed YOLOv8s Held-Out Precision–Recall Plot; All-Class AP50 Is Approximately 0.590	35
Figure 5.5	Notebook Validation Precision–Recall Trade-off across Confidence Thresholds (YOLO11s, 512 Pixels)	36
Figure 5.6	Actual Clean Road Photograph and Fresh ONNX Inference Executed on 14 October 2026	38

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
Table 2.1	Representative Published Work and the Proposed System	10
Table 3.1	Mapping from Score to Priority Band and Map Colour	15
Table 4.1	Principal Components and Their Responsibilities	18
Table 4.2	Prepared Dataset Partitions and Class Instances	19
Table 4.3	Data Preparation Steps and Their Limits	21
Table 4.4	Recorded Training Configuration	22
Table 4.5	Inference and Upload Constants	23
Table 4.6	Frontend Source Modules and Their Responsibilities	24
Table 4.7	Storage Modes and Their Behaviour	26
Table 4.8	Review Status Labels	27
Table 4.9	Principal Routes of the Optional Python Backend	28
Table 4.10	Evidence Used in This Report	30
Table 5.1	Held-Out Test Metrics of the Deployed YOLOv8s Checkpoint	33
Table 5.2	Class-Wise Held-Out Results of the Deployed Checkpoint	34
Table 5.3	Confidence Threshold Experiment on the YOLO11s 512-Pixel Notebook Checkpoint	36
Table 5.4	Notebook YOLO11s Test Results (Separate from the Deployed Model)	37
Table 5.5	Class-Wise Notebook Results at 640 Pixels	37
Table 5.6	ONNX Predictions for the Clean 512 × 512 Image	38
Table 5.7	Additional Runtime Checks and Rule Outputs	38
Table 5.8	Recorded Software Test Outcomes	39
Table 5.9	Implementation and Verification Status of the Project Features	40

LIST OF SYMBOLS AND ABBREVIATIONS

RDD2022	- Road Damage Dataset 2022 (multi-country image dataset)
CRDDC	- Crowdsensing-based Road Damage Detection Challenge
D00	- Longitudinal crack class
D10	- Transverse crack class
D20	- Alligator crack class
D40	- Pothole class
YOLO	- You Only Look Once (single-pass object detector family)
YOLOv8s	- Small variant of the YOLOv8 detector
CNN	- Convolutional Neural Network
ONNX	- Open Neural Network Exchange (portable model format)
WASM	- WebAssembly (browser runtime used by ONNX Runtime Web)
PWA	- Progressive Web App
GPS	- Global Positioning System
IoU	- Intersection over Union
NMS	- Non-Maximum Suppression
AP	- Average Precision
mAP	- Mean Average Precision
mAP50	- Mean average precision at IoU 0.50
mAP50–95	- Mean average precision averaged over IoU 0.50 to 0.95
TP, FP, FN	- True positives, false positives, false negatives
F1	- Harmonic mean of precision and recall
DFL	- Distribution Focal Loss
SGD	- Stochastic Gradient Descent
SHA-256	- 256-bit secure hash algorithm used for checkpoint and image identity
IndexedDB	- Browser database used for local inspection records
B, A, K, C, S	- Priority terms: class baseline, area coverage, count term, context term, score

CHAPTER 1
INTRODUCTION

1.1 Background and Evolution of Road Damage Detection

This report provides background information and an evolution of RDDP.This report gives background details and the evolution of RDDP. The maintenance agencies must understand where and what caused the failure of the road networks which were caused by traffic loading, weather and ageing. However, conventional condition surveys rely on trained inspectors or survey vehicles and the applications for computer vision concepts for civil infrastructure have been the subject of many studies concerned with automating the detection of defects and condition assessments of asphalt and concrete surfaces [1]. An alternative of lower costs would be through smartphone sensing. The researchers were able to detect real time potholes using the accelerometers from an Android phone in early works [2].

Whereas, in the past, it was only about deciding whether there was damage or not, this vision was redefined to identifying type and location of damage in the image back by deep learning. Maeda et al. gathered a dataset of 163,664 road images (1,500+ km driven with 7 Japanese municipalities), to which they added 9,053 damage instances (15,435 detected for the 8 damage types) to train SSD-based detectors on 8 damage types [3]. Arya et al. adapted the research to Indian and the Czech Republic's road images and proved that a simple Japanese image trained model does not work well in road images of the other two countries [4].

This direction was brought together in the RDD2022dataset. It has 47,420 images collected from six countries (Japan, India, the Czech Republic, Norway, the United States and China) and more than 55,000 instances of 4 damage types [5] are annotated. It is made available for the Crowdsensing based Road Damage Detection Challenge (CRDDC2022) for which over 60 teams from 19 countries registered and the top solution was an ensemble of models of the family of the YOLO and Faster R-CNN with an F1 score of 76% on the test data (across all six countries) [6]. Further, the four types of damage dealt are: Longitudinal Cracks (D00), Transverse Cracks (D10), Alligator cracks (D20) and potholes (D40), which are the five classes in this project. Boxes and scores are created on only one image from a detector,

however. If there is a "maintenance viewer", then he or she will require the original evidence as well, a "maintenance status" (which it also has), and a "defensible prioritization of which evidence to review first. This project is based on the following workflow around a trained detector. The problem is given in more detail in the next section.

1.2 Problem Statement

Road inspection does not start with the visible part of the failure, but from documenting some degree of failure and provides a defence with the interpretation. There are three problems that need to be solved, these are outlined below and account for some of the design decisions made later in the report.

1.2.1 Detection without Evidence or Location

It's hard to duplicate consistently within numerous photographs from manual review of images. An image-only classifier cannot indicate the location of a fault and a detector without storage info (with no memory) provides no connection to future checks. The engineering challenge is to keep the output of the model, under close control; to associate that model with the capture information which is available; to make it accessible via a map, history and reports.

1.2.2 No Transparent Basis for Ordering Records

The boxes and confidence scores are not indicative of any imaginary or real priory. Prediction confidence is not related to the severity of the risk but to the degree of confidence that the model has. image detector cannot directly provide the ratings for distress type, distress severity and distress quantity as is done in the Pavement Condition Index procedure [7] as it is a part of the normal pavement condition practice. In addition, there needs to be an easy-to-reproduce screening rule that can be explained and audited that is not an engineering severity assessment, and a clear declaration that it's not.

1.2.3 Reliability and Honest Reporting

One detector that fails to detect the defect or states that the defect has been detected but is not present could fool a reviewer if all predictions are shown to have been confirmed as damage. The research problem is separate: Given the detector is able to detect damage that is not

visible in the image, is its ability to do so sufficient for assisted screening. There needs to be repetition of the evaluation but not by holding students out, reporting scores by class, and definite restrictions. A failure to load a model to a display would be a poorer user interface than a non-user interface, which means that predictions in this project go first as "awaiting human evaluation", and failures get reported as "failure".

1.3 Motivation for Unified Risk-Based Maintenance Prioritisation

The project is not a four tool workflow for detection, prioritisation, evidence, and location, but is a joined up approach. Only one inspection yields boxes and classes, a screening priority, Memoized images of the original and annotated image, and (if any) a marker on a map to work from. Multi-hazard is synonymous with damage classes (Four) that a damsite has been trained on, and risk is synonymous with screening rule that a dam site is documented, not a validated risk model. There are two additional concepts in the design. The first is in regard to deployment. If the detector is run on the browser via ONNX Runtime Web [8] then there is no need for its own hosted Python detection server and guest records will remain on the device unless some optional synchronisation is performed. The second is with respect to restraint. The priority rule means that it gives no weight to the degree of confidence in a prediction, and it is not used to quantify danger, but is made available as a practical aid for ordering records for humans to assess.

1.4 Objectives of the Project

The project seeks to create and test an inspection app based on the road damage detector that has been trained with four classes of road damage; to report what has been achieved; and, to report what still has not been verified. The goals are:

1.Detect: Report class and box co-ordinates of all predictions accepted; all reliable trained weights, trained by 1000 step execution, are applied to detect these classes and co-ordinates.

2.Direct access: To have an image-upload path and camera path that don't require the user to have an account.

3. Process uploads with proper location: Acknowledge located / unlocated inspection by having the user select/deny a location when asking for a picture or GPS location; Keep the capture time location with live positive detections.

4. Make it clear what things are important: to create a practical and repeatable maintenance screening score and show the explanation for the score.

5. Preserve evidence: Preserve original evidence, model settings and review history locally and synchronise it optionally.

6. Assess and limit claims: Quantitatively assess and document limitation of detection (but do not report all predictions as "confirmed defect").

1.5 Scope and Operational Boundaries

There are four categories of detector that identify each as follows: D00 (longitudinal cracks), D10 (transverse cracks), D20 (alligator cracks) and D40 (potholes). These absenteeism categories have not been turned on during the construction of the dataset and these are not covering all potential sources of pavement damages, but only selected damages. The model neither defines pothole depth, crack width nor road structural capacity or hazards conditions beyond its classes of training nor does it define the training classes used. The contribution is built and ability to see all the inspection processes transparently, around a trained detector, as integrated applications. It's not a new YOLO architecture nor an autonomous driving system and nor, of course, is it a model of geographic localisation which is then validated by engineers. Evaluation is performed on the split of prepared RDD2022 data marked with labels.Evaluation employs labelled held-out splitted dataset of the data properly prepared for RDD2022. The application of four limitation is given, four times repeated in Chapter 6. This test split has everything other than a route-disjoint/WTST. Typical on-the-road acceptance (complete physical-device acceptance) is not proven. This is not validated against engineer assigned severity - priority rule. None of CONFIGURATION_TO_DELIVER is configured. The ability to retain these limits visible is given by the features in this report, which are provided with one of three labels: implemented and checked, implemented but not physically verified, not implemented. The labels have been gathered in one place in table 5.9.

1.6 Organization of the Report

Published research is summarised in Chapter 2 looking at road damage datasets, detectors used to identify the damages and evaluate/condition assessment model. The gap analysis between the published works and the project is summarised. The mathematical formulation of the outputs is given in Chapter three of the image, outputs of the detector, the objective that is to be recorded during training, overlap and filtering, evaluation measures, its priority in a screening and revision rules for location. The architecture and implementation, such as detector, Browser inference, the database dataset, the camera behaviour, the storage, reporting and reproducibility is detailed in chapter 4. The results measured and recorded software checks are presented in Chapter 5. The contribution is summarized in chapter 6, the limitations are outlined and suggestions for further work are made. The references follow.

CHAPTER 2

LITERATURE REVIEW

This chapter summarizes the published research in five parts: road damage data sets and benchmarks; the architectures of the detectors and the desired training goals; process of using the smartphone devices for the detection; and data preparation and assessment; condition assessment. This ends with the shortfalls in research that are addressed in this project. The review does not cover all the sources but is limited to the ones cited in the references.

2.1 Road Damage Datasets and Benchmarks

Maeda et al. [3] designed a large public dataset which was created by a collaboration with seven municipalities in Japan by taking images of the car dashboard from a smartphone. It features 9,053 annotated images and includes a total of 15,435 instances of damage in them, which have been extracted from 163,664 images collected after a flight path of approximately 1,500 km. The authors note that previous research primarily identified damage (or its absence) but not damage type, and that the previous research had utilized private datasets, making comparisons difficult. They developed SSD detectors by the Inception V2 and MobileNet backbones, evaluated their accuracy and speed on a GPU server and on a smartphone and finally combined to recognize 8 types of damage, based on Japanese maintenance guidelines. The dataset was expanded to 26,620 images from Japan, India and the Czech Republic by Arya et al. (2008) [4] where four classes – D00, D10, D20, D40 – were considered. They were able to test 16 SSD MobileNet models in 30 different experiments and came to these results: A model trained with Japanese data was quite poor when tested on roads in India and Czech, if they could be tested at all; In general, more generalisation was achieved when the model was trained with multiple data sources, and including local data with Japanese data produced better results for generalisation; Transverse cracks were found to be more difficult to identify in India, as there were few examples in Indian data. RDD2022 [5] made the series into 47,420 images from six countries (Japan,

India, the Czech Republic, Norway, the United States and China) while having over 55,000 instances of damage annotated for the same set of four damage types. It was published for the IEEE Big Data Cup challenge “CRDDC2022” whose evaluations were carried out online on representations provided by test images that were not included in the training set. Ranked by F1 score on the last unseen test images on an online evaluation platform [6]. Over 60 teams from 19 countries entered into competition, the top 11 teams were reported (the exact number was not stated) and the winning team used a combination of two of the families, YOLO and Faster R-CNN to achieve 76% F1 score on test data from all six countries.

2.2 Object Detection Architectures and Training Objectives

The feature of being able to operate in real time had been enabled by the detection trade-off, suggested by YOLO [9] to be a single regression pass over the image. However, faster R-CNN [10] first generates regions then does the classification; and SSD [11] makes classifiers predictions based on default boxes in multiple feature-map scales in one pass. YOLOv7 [12] was one such later version of YOLO which gained from the optimisations done during training, as well as its enhanced network. To reduce computation, but maintain accuracy, ideas were introduced to cross-stage partial networks [13] that split feature maps across stages. Recent backbones for the YOLO model are inspired by these concepts. For the detector used here there are two loss ideas which are relevant. Some losses that enhance the overlap measure for bounding-box regression, such as Distance-IoU and Complete-IoU losses [14] are designed for this purpose. Combining the localization quality in the class prediction, as in generalised focal loss [15] which takes a different approach to box location and represents it as a distribution instead of a single value, this requires training a continuous form of the focal loss. The Ultralytics YOLOv8 framework is based upon these concepts and has a series of detectors with five different sizes: a distributional box term (also called distribution focal loss) is logged in the training progress. In this late model YOLO11 [17] an overview of new blocks (C3k2, SPPF and C2PSA) and model sizes from nano to extra-large is provided.

2.3 Smartphone Sensing, Deployment and Mapping

Road conditions have been detected in a camera-free manner. Later, Mednis et al. [2] generated pothole detection system depending on the accelerometers of Android telephones. Event detection senses are useful for detecting an event with the phone's location, however an image alone doesn't tell where the defect is in the image. Place detection is part of condition assessment, which is covered in computer-vision-based defect detection and condition assessment for asphalt and concrete infrastructure by Koch et al. [1]. A trained detector can be packaged into ONNX format for deployment, and then run using ONNX Runtime [8] with a browser port. Maeda et al. [3] had earlier demonstrated a comparison of a GPU server with a smartphone for inference, and showed that of course the location of inference is a design decision. Most initial map presentations in web tools are based on the OpenStreetMap package of map tiles [18]; access to the network is needed to get freshly downloaded tile packages unless they are maintained using a license.

2.4 Data Preparation, Leakage and Evaluation

From the cross-country results in [4] it is seen that where the model is trained and where it is tested is important when studying the actual behaviour of the model, hence the choice of the test split is essential. Resemblance of images within training and testing set leads to inflated scores. preciso duplicates can be obtained by using a cryptographic hash function for the decryption of the image, like the sha256hash [19] and near-duplicates are obtained by using a perceptual hash function for the decryption of the image, which are the subject of study and design in [20]. Both are not always successful in locating a near duplicate and neither can retrieve lost information, e.g., the section of road a photograph has been taken from. Overlap-based measures are used for reporting detection results. The Intersection over Union metric (and its generalised version of [21]) quantifies the "goodness" of a prediction as its intersection with a reference box, quantified in terms of the surface area of the overlap. The COCO benchmark [22] popularised average precision for a range of IoU thresholds (0.50, ..., 0.95) and the CRDDC, challenge protocols, operate with a fixed overlap rule (CI) using F1 [6]. A non-maximum suppression step [23] eliminates redundant boxes as detectors generate

many overlapping candidates. None of these measures is classification accuracy and that fails to tell if they are read without its recall.

2.5 Condition Assessment and Maintenance Prioritisation

The Pavement Condition Index was adopted in ASTM D6433 [7] to practice the measurement of the quality of pavement by determining the type, severity and number of each distress observed. A detector focused on four classes provides image type and location, not measurements like "how wide a crack is" or "how deep a pothole is. The challenge paper [6] sorts entries according to the detection F1 score and providing an order for detection during the maintenance is not part of the evaluation process described there. In [1] the detection problem is considered as part of a larger condition-assessment problem.

2.6 Critical Synthesis and Identified Research Gaps

The three in the reviewed work are summarized below. There is now public multi-country data [5] and a clear measure of the damage from roads [6]. Most commonly, it is used with the YOLO family of single-pass detectors [9, 12, 16]. Evaluation is reported and only little attention is paid to the surrounding inspection workflow in these resources. This is followed by six gaps, and the project addresses each gap in a partial and deliberately delineated manner.The project addresses each gap in a partial and deliberately bounded manner and a total of five gaps follow. Benchmark papers 1. Detection without a record: score detections on images 1st grade [5] and 2nd grade [6]. The maintenance reviewer's memory requires the sources described here to contain evidence and model settings and the results of reviewing history. No detecting ordering transparency, condition indices [7] depend on the surveyed severity and quantity, which does not get provided by a four-class detector. A simple (and documented) screening rule, derived from the detector outputs, is required, but should be labeled as 'unvalidated'. 3. Location: the device records events, as done in [2], locally, by georeferencing it. The alone property of imaging doesn't enable one to give the position of a defect, hence location handling needs to be explicit of what GPS is and what it is not able to establish. Leakage and honest evaluation: near-duplicate control [20]: there is often overlap between partitions that can be reduced, but is still not eliminated, using near-duplicate control. Route identifiers are often unavailable. The result, in turn, should therefore be given

in terms of the split protocol + protocol boundaries. For deployment without a server, it is possible to have an exported model executed in the browser [8] which eliminates the requirement for a hosted inference service, but implications for storage, use without Internet connection and synchronisation need to be worked out and tested. The project implements one browser-deployable inspection workflow around a YOLOv8s detector, which includes four classes, a clear screening score (without confidence, explainable to the reporter), evidence-preserving records with review marks (clearly showing if changes were made or not), yet explicit location rules, and the separation of measured outcomes and unverified behavior in the report. Table 2.1 compares the project with two of the most recent projects that are closest to it. The numerical data in this table have been gathered from various protocols and cannot be directly compared.

Table 2.1: Representative Published Work and the Proposed System

	Maeda et al. [3]	**CRDDC-2022 top solutions [6]**	**Proposed**
Data	9,053 annotated Japanese smartphone images, 8 damage types [3]	RDD2022: 47,420 images, 6 countries, 4 types [5]	Prepared RDD2022 subset: 35,490 retained images, 4 classes
Detector	SSD with Inception V2 and MobileNet [3]	Ensembles of YOLO and Faster R-CNN family models [6]	YOLOv8s checkpoint exported to ONNX, run in the browser
Reported score	Accuracy and speed compared on GPU server and smartphone	Best F1 of 76% on pooled test data from six countries	Held-out mAP50 59.02%, mAP50–95 30.51% (own split)
Priority	Not described in the abstract	Not covered	Rule-based screening score S, confidence excluded
Evidence and location	Not described in the abstract	Not covered	Stored original and annotated images; optional GPS; map
Comparability	Different dataset	Different split and metric	Scores are not directly comparable

CHAPTER 3
MATHEMATICAL MODELING & THEORETICAL FORMULATIONS

This chapter establishes inputs and outputs of the detector, the letterboxing and decoding process, a recording of the goal of training, overlap and filtering, screening priority and rules of location and record revision. Symbols are defined the first time they are used. If a numerical value is quoted it is that of the setting that has been set to the deployed system and should not be considered as derived here.

3.1 Image Model and Detector Output

Suppose the image is an RGB image I, with width W and height H. The set of detections is estimated by the detector

$$D = \{(b_i, k_i, p_i)\}_{i=1}^{N} \quad \textbf{(3.1)}$$

where bi = (x1, y1, x2, y2) is a box in original image coordinates, ki is one of the four class indices (D00, D10, D20 or D40) and pi ∈ [0, 1] is the associated prediction score. There are many possible defects in this image, so N may take on many values, and N = 0 may only occur if a run of inference actually took place. The score pi is the prediction model's confidence in the prediction. It is not about the chance that a road crash is going to happen nor is it a indicator of the damages.

3.2 Letterboxing and Output Decoding

The input of the network is a 640×640 tensor. The image is scaled so as to not stretch the defect shapes one way whilst not in the others, so scaling it uniformly. The scale is

$$r = \min(\, 640 / W,\ 640 / H \,) \quad \textbf{(3.2)}$$

This resized image is then equalized in size with the value of 114 to the shape required as illustrated in Figure 3.1. Have the padding offsets dx and dy as horizontal and vertical respectively. If a box is specified in model coordinates, e.g., (x', y'), then this box in model-coordinates is converted back to the original-image coordinates.

$$x = (x' - d_x) / r, \quad y = (y' - d_y) / r \tag{3.3}$$

This is then clipped to the size of the image. The model output has shape [1, 8, 8400]: four box values and four class scores for each of 8,400 candidate locations, which equals $80^2 + 40^2 + 20^2$ for a 640 × 640 input.

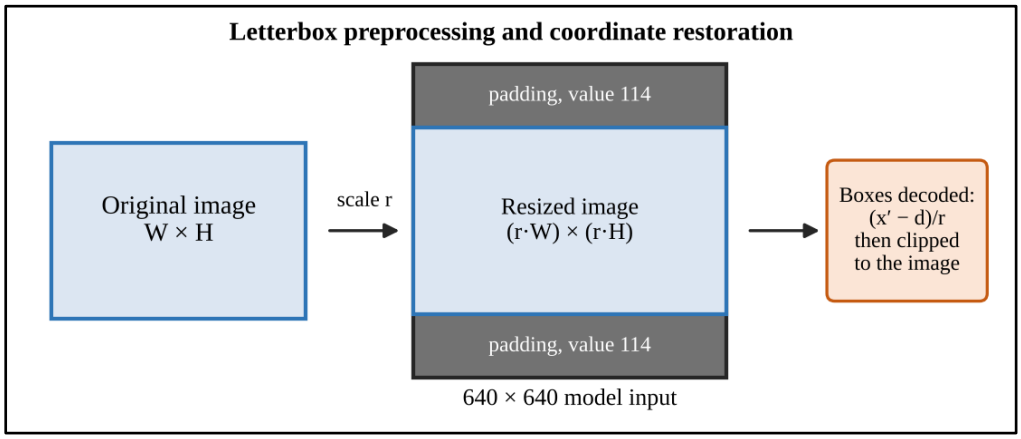


Figure 3.1: Letterbox Preprocessing and Coordinate Restoration

3.3 Training Objective

The saved Ultralytics training log contains three loss components: box, classification and distribution focal loss. Conceptually, it is a formulation of both localization, category prediction and box-distribution learning, given with weights, and is averaged out.Conceptually, the goal is a mix of localisation, category prediction and box-distribution learning, weighted and averaged.

$$L = \lambda_{box} L_{box} + \lambda_{cls} L_{cls} + \lambda_{dfl} L_{dfl} \tag{3.4}$$

T is the box term of the [14],[21] class of regression losses with an overlap heuristic which can be implemented in Ultralytics framework [16] and f(x) is the distribution term of [15] with distribution focal loss (DF). No function is claimed on the report which does not say the numerical values for the weights λ because those weights are not saved on the report and did not appear in the results here.

Training used stochastic gradient descent with momentum. With momentum μ = 0.937, weight decay w = 0.0005 and learning rate η (initially 0.01), one update has the form

$$v \leftarrow \mu\, v + \nabla L(\theta) + w\, \theta, \quad \theta \leftarrow \theta - \eta\, v \tag{3.5}$$

The checkpoint was selected by validation fitness (best.pt) and not by the final epoch or by training loss. Section 5.2 reports how the logged losses behave.

3.4 Overlap, Confidence Filtering and Non-Maximum Suppression

Intersection over Union measures the overlap of two boxes A and B, whether a prediction and a reference box or two competing predictions:

$$\mathrm{IoU}(A, B) = \mathrm{area}(A \cap B) \,/\, \mathrm{area}(A \cup B) \tag{3.6}$$

Confidence filtering retains predictions whose score is at least the selected threshold τ, and the application default is $\tau = 0.25$:

$$\text{keep detection } i \text{ if } p_i \geq \tau \tag{3.7}$$

Class-aware non-maximum suppression [23] then removes redundant detections. A lower-scored box j is suppressed when a higher-scored box i of the same class overlaps it by more than the configured threshold, which is 0.70 in this project:

$$\text{suppress } j \text{ if } k_i = k_j,\ p_i > p_j,\ \mathrm{IoU}(b_i, b_j) > \theta_{nms} \tag{3.8}$$

At most 300 predictions are kept per image. Suppression works within one image. It does not deduplicate the same physical defect across successive camera frames, a point that returns in Section 4.5.

3.5 Evaluation Metrics

But only a detection made by the class is a true positive and a detection that overlaps a reference box so that it matches the overlap rule with the reference box. There are “false positives” and “false negatives.” False-positive are cases in which a new vector is correctly predicted but the reference box is missing, and false negatives are cases where a new vector is not matched to the reference box. Precision P & Recalls R are

$$P = TP \,/\, (TP + FP), \quad R = TP \,/\, (TP + FN) \tag{3.9}$$

and the F1 score is their harmonic mean:

$$F1 = 2PR \,/\, (P + R) \tag{3.10}$$

Average precision summarises the precision–recall curve of one class, and mean average precision averages it over the four classes:

$$\mathrm{AP_k} = \int_0^1 \mathrm{P_k}(\mathrm{R})\,\mathrm{dR}, \quad \mathrm{mAP} = (1/4)\sum_k \mathrm{AP_k} \tag{3.11}$$

mAP50 evaluates the matching at IoU 0.50. mAP50–95 averages the mean average precision over the overlap thresholds 0.50 to 0.95 in steps of 0.05, following the convention of [22]:

$$\mathrm{mAP50{-}95} = (1/10)\sum_{t=0.50,0.55,\ldots,0.95} \mathrm{mAP_t} \tag{3.12}$$

Neither mAP50 nor mAP50–95 is classification accuracy. A precision value must be read with its recall, a point that matters for the threshold experiment in Section 5.5.

3.6 Unified Screening Priority Formulation

Priority is assigned by an academic rule and not by a separately trained severity model. For an image with N accepted detections, the score is

$$\mathrm{S} = \min(\,100,\ \mathrm{round}(\,\mathrm{B} + 20\mathrm{A} + \mathrm{K} + \mathrm{C}\,)\,) \tag{3.13}$$

B is the baseline of the highest-baseline class present,

$$\mathrm{B} = \max_i \beta(\mathrm{k_i}) \tag{3.14}$$

with baselines β(D00) = 15, β(D10) = 20, β(D20) = 35 and β(D40) = 45. A is the summed bounding-box area as a fraction of the image area, capped at 1:

$$\mathrm{A} = \min(\,1,\ \textstyle\sum_i \mathrm{area}(\mathrm{b_i}) \,/\, (\mathrm{W}\cdot\mathrm{H})\,) \tag{3.15}$$

K rewards multiple detections and is capped at 15:

$$\mathrm{K} = \min(\,15,\ 3(\mathrm{N}-1)\,) \tag{3.16}$$

C collects the context inputs importance m, traffic t and urgency u, each configured from 1 to 3:

$$\mathrm{C} = 5(\mathrm{m}-1) + 5(\mathrm{t}-1) + 5(\mathrm{u}-1) \tag{3.17}$$

Prediction confidence is deliberately excluded, because confidence is not danger. The score is mapped to a priority band and a map colour as in Table 3.1. With no detections the rule returns zero together with a no-detected-damage explanation, which is not a road-safety certification. Figure 3.2 summarises the calculation.

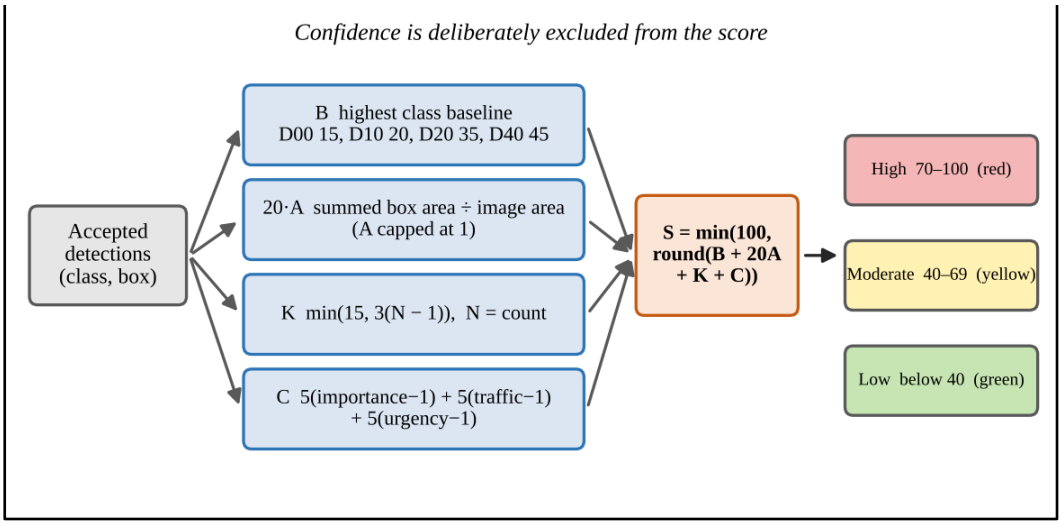


Figure 3.2: Screening Priority Calculation. Four Terms Are Added, Rounded, Capped at 100 and Mapped to a Band; Confidence Does Not Enter

Table 3.1: Mapping from Score to Priority Band and Map Colour

Score	Priority	Map colour
Below 40	Low	Green
40–69	Moderate	Yellow
70–100	High	Red

There are 3 properties of the rule that to be stated. Boxes could sometimes overlap to make the same area count multiple times, which is why the total coverage could be more than what is expected – A may overestimate the total effect. The number of pixels covered is dependent on the camera's perspective, and will not equal pavement area or crack width or pothole depth. Last but not least, there is no rating of severity against the actual accident results or damage repair expenses, nor is there a maintenance program to follow. It's designed to compile data for retrieval by people. In Section 5.7 a worked numerical example is provided.

3.7 Location, Retry and Revision Rules

The application encodes several rules as explicit conditions. A location is valid only if its coordinates are finite and lie in the permitted ranges for latitude φ and longitude λ:

$$-90 \leq \varphi \leq 90, \quad -180 \leq \lambda \leq 180 \tag{3.18}$$

The live-camera save policy stores a frame only when it holds at least one accepted detection and a valid location fix that is no more than 10 seconds old at capture. The fix values are copied before inference so that a later GPS update cannot change the position saved with that image:

$$\text{save frame} \Leftrightarrow N_{acc} \geq 1 \wedge (t_{capture} - t_{fix}) \leq 10\ \text{s} \wedge \text{location valid} \quad \textbf{(3.19)}$$

The user-selected pause of 1, 2 or 5 seconds is additional to inference time, so the interval between analysed frames is

$$T_{cycle} = T_{inference} + T_{pause} \quad \textbf{(3.20)}$$

which is not a guaranteed frame rate. Records follow two further rules. An edit carries the revision it expects, and it is accepted only if that revision is still current, after which the stored revision increases by one:

$$\text{accept edit} \Leftrightarrow r_{expected} = r_{stored},\ \text{then}\ r_{stored} \leftarrow r_{stored} + 1 \quad \textbf{(3.21)}$$

A request that reuses a client identifier is compared by fingerprint: identical content returns the earlier result and different content is reported as a conflict, while an identifier that was deleted cannot be replayed as a new record. These rules prevent duplicate submissions and silent overwrites. They do not merge different camera frames that show the same road defect.

CHAPTER 4

SYSTEM ARCHITECTURE AND IMPLEMENTATION

Details the design of the system are given in this chapter. It includes the architecture, the prepared dataset and its partitions, the detector, its training record, the browser, and the behaviour of the former, the Detector, the camera and the map, the storage, the reporting and the repository structure and (opcional) the reproduction of the work, done on the optional back end. The descriptions with the verification status are provided for each feature.

4.1 Platform Architecture Overview

RoadGuard AI features an AI trained road damage detector that is connected to a road inspection website. It uses React and TypeScript as the web-components, serves it hosted in Vite, maps with Leaflet, local persistence with Dexie and IndexedDB, inference with ONNX Runtime Web [8] and local report generation with jsPDF. There are two different ways of deployment—with optional cloud synchronisation and Python backend. Just one inspection is shown in Figure 4.1.

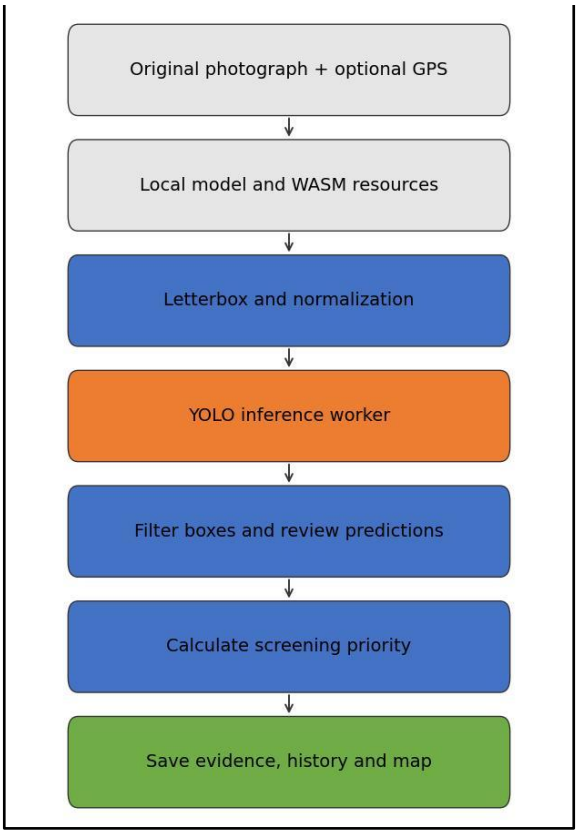


Figure 4.1: System Architecture. Explanatory Implementation Pipeline from Photograph to Saved Record; Review Can Occur after Initial Saving

This diagram is descriptive of the flow of data only and is NOT a measured experiment. Records that are placed can be shown on the map and anytime a record is not located they will be kept in the history. No requirements, except for optional Python service, for the default browser workflow. The Principal/Commissioners and relevant departments are shown in Table 4.1.

Table 4.1: Principal Components and Their Responsibilities

Component	Responsibility
React interface	Capture or upload, status, result presentation and user actions
Inference worker	Local ONNX execution and decoded predictions
Priority service	Deterministic screening score
Storage adapter	Evidence, records, drafts, mutations and query results
Optional services	Account synchronisation, or Python inference and persistence

4.2 Datasets and Data Pipeline

A collection of prepared RDD2022 is used in the deployed detector [5]. Target annotations were kept – images that did not have target annotations were not included for supervised training and evaluation – and images acquired only for the lack of supported categories were not included. Empty annotation files that contained valid annotations were retained as target-negatives. Figure 4.2 presents the partitions and class instances obtained and Table 4.2 shows the classes obtained after the partition and Figure 4.3 presents samples of the ground truth annotation from the source dataset.

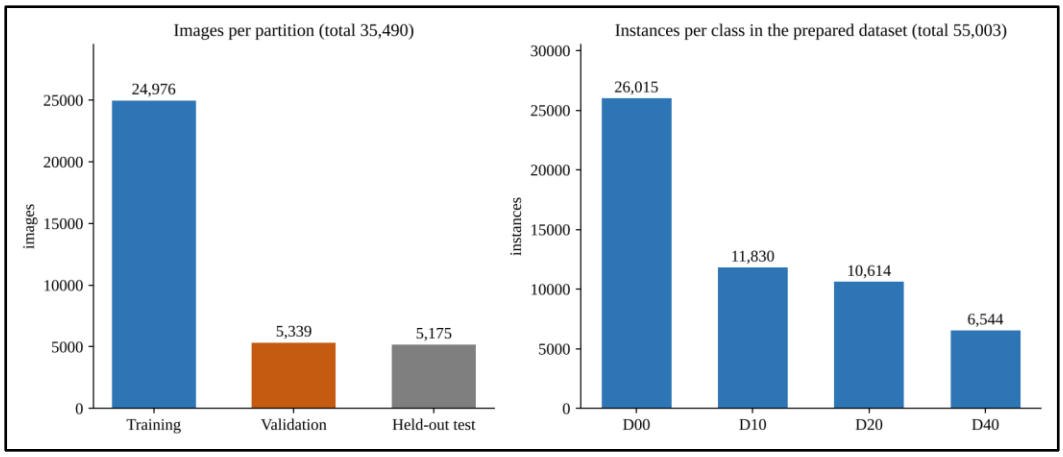


Figure 4.2: Dataset Composition. Left: Images per Partition. Right: Instances per Class in the Prepared Dataset

Table 4.2: Prepared Dataset Partitions and Class Instances

Item	Count	Share
Training images	24,976	70.4% of images
Validation images	5,339	15.0% of images
Held-out test images	5,175	14.6% of images
Total retained images	35,490	100%
D00 instances	26,015	47.3% of instances
D10 instances	11,830	21.5% of instances
D20 instances	10,614	19.3% of instances
D40 instances	6,544	11.9% of instances

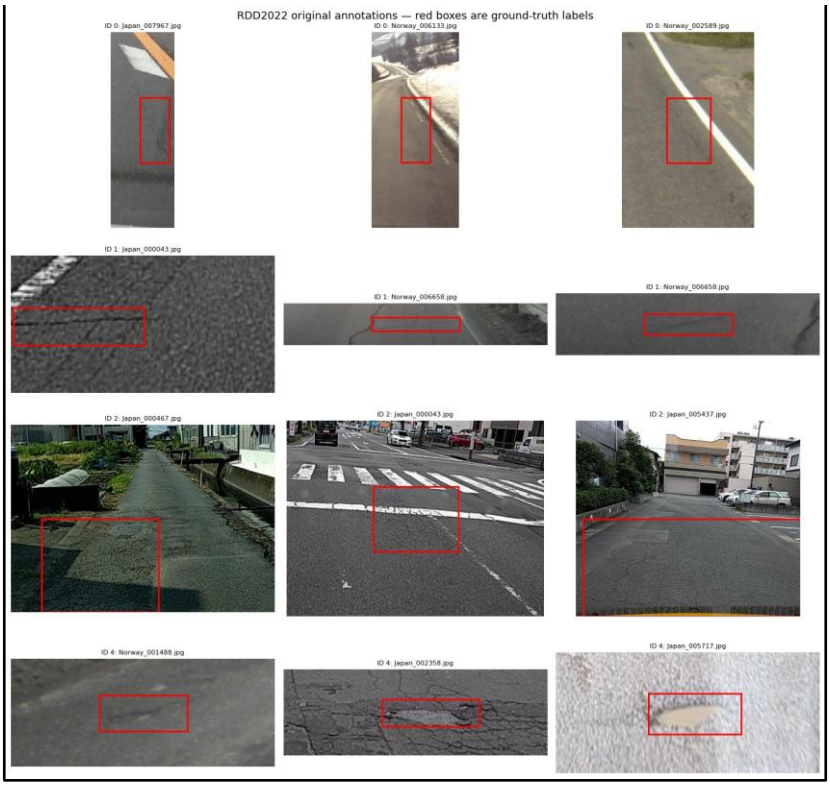


Figure 4.3: Sample RDD2022 Ground-Truth Annotations (Red Boxes) for the Four Retained Target Classes; Rows Show Original Class IDs 0, 1, 2 and 4

The numbers in Table 4.2 are percentages of the actual numerators. Class distribution is not equal, as in the case of D40 potholes, the number of occurrences is least, with the former longitudinal crack being most, which is explored with the class wise results in section 5.4 without attributing a cause to that.

4.2.1 Partitioning and Leakage Controls

Before allocation, images were divided into sets of near- or exact-duplicates by two types of hashing algorithms: Decoded-image SHA-256 [19], and same-country perceptual [20]. This recorded protocol called the number 64/4 Hamming 42 was with 64-bit perceptual hashes (up to Hamming distance of 4), an allocation target set as 70/15/15, and seed 42. The last column in table 4.2 gives the final count which represents the allocation. The procedure for preparing the food and the limit associated with each is given in Table 4.3.

Table 4.3: Data Preparation Steps and Their Limits

Step	Recorded protocol	Limit
Label filtering	D00, D10, D20 and D40 retained; unsupported categories in mixed images ignored	Other hazards are not modelled
Image exclusion	Unannotated and unsupported-only images excluded; valid empty annotation files kept as target-negative	Excluded images are not evaluated
Duplicate grouping	Decoded-image SHA-256 plus same-country 64-bit perceptual hash, Hamming distance up to 4	Not every near-duplicate can be found
Allocation	70/15/15 target, seed 42; final 24,976 / 5,339 / 5,175 images	Not route-disjoint, because route identifiers were unavailable
Test split	Held out from the available official training annotations	Not an independently collected field test

It is not possible to record and return all the near duplicate images. The test set used for the task has been selected from available official test annotations and is NOT an independent field test. Selected pavement damage was included in the four classes, the remaining damage categories were not included, and the model could not set the depth of a pothole, or the width or structural capacity of a crack.

4.3 Detector Implementation and Checkpoint Integration

The model used in the detector deployed is YOLOv8s [16] from the Ultralytics family of YOLO models. It is made up of three parts: the structure of this model family is depicted in Fig. 4.4 and consists of a convolutional backbone, a feature-fusion neck and a decoupled

head with three outputs, a box output and a class output, as well as an optional filtering component provided by the application itself. The figure represents a schematic rather there being a measured figure.

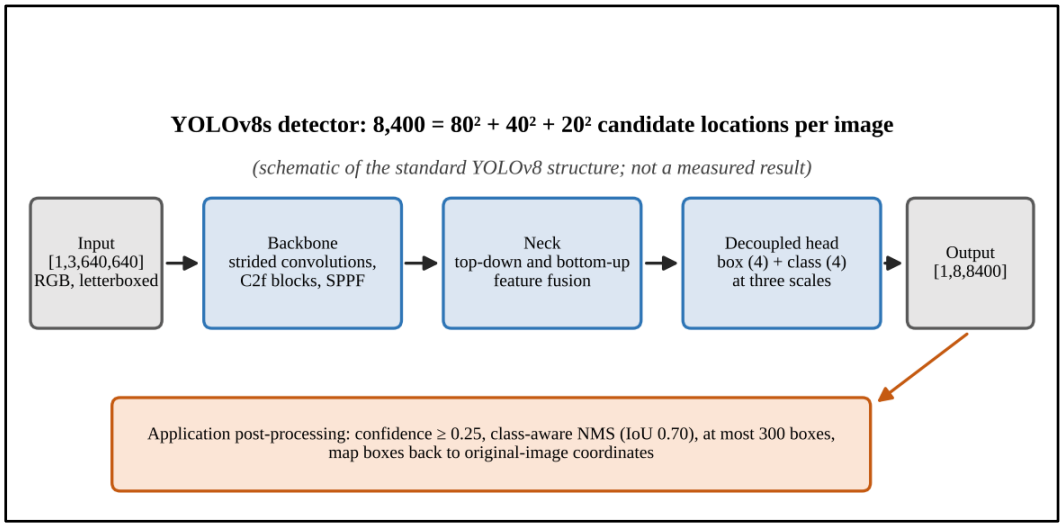


Figure 4.4: YOLOv8s Detector Structure and the Application's Post-Processing (Schematic)

Table 4.4 lists the training settings recorded in the model metadata.

Table 4.4: Recorded Training Configuration

Training setting	Recorded value
Architecture	YOLOv8s
Input size	640 × 640
Batch size	32
Optimiser	SGD
Initial learning rate	0.01
Momentum / weight decay	0.937 / 0.0005
Seed / patience	42 / 15
Requested epochs / time budget	80 / 8 hours
Checkpoint selection	Validation fitness; best.pt

The logged epochs are contained in the actual results CSV which has 91 epochs. Great care should be taken to distinguish the requested configuration from the recorded duration as the

duration was constrained by using time budgeting for the run. The maximum mAP50–95 achieved is at epoch 87, and the epoch-log values are not reported as a checkpoint.

4.3.1 Real Checkpoint Integration

A notebook which was 75.6% identical to the supplied notebook, but was missing the checkpoint bytes, was submitted.A copy of the supplied notebook was submitted, minus checkpoint bytes. The recovered YOLOv8s checkpoint is verified, converted to ONNX and packaged for the website and its corresponding SHA-256 checksum is noted in Section 4.7. The model is run locally by ONNX Runtime Web, eliminating the requirement to have the detection server provisioned separately in Python. The record labelled 'Notebook experiments' is a separate record, and is not the model analysed in the website (Section 5.6). There is no retraining involved with model integration, nor is it an improvement in detection accuracy. The exported graph and the PyTorch checkpoint have been analyzed on real images, which is detailed in Section 5.8.

4.4 Browser Inference and Priority Implementation

Images are padded with value of 114, converted to RGB. The default confidence filtering threshold is 0.25, the class-aware non-maximum suppression IoU is 0.70 and 300 predictions are kept at most of the times. Boxes are converted to image-coordinates. A worker-mode inference was implemented and computations are done in a worker (so interface rendering is separate), and a hosted worker-loading path was fixed up during the project. The constants are given in table 4.5.

Section 3.6 is a deterministic service, which has the following priority rule: The modelCore.ts file includes the priority-related browser utilities with the preprocessing and decoding; its formula is in risk.py. The list of frontend modules and what they do, is looked at in table 4.6.

Table 4.5: Inference and Upload Constants

Constant	Value	Purpose
Input tensor	[1,3,640,640]	RGB image after letterboxing
Output tensor	[1,8,8400]	4 box values and 4 class scores for each of 8,400 candidates
Padding value	114	Letterbox fill
Confidence threshold	0.25	Default filter on prediction scores
NMS IoU threshold	0.70	Class-aware suppression of overlapping boxes
Maximum predictions	300	Upper limit per image
Upload limits	10 MB, 25 MP	Accepted JPEG, PNG or WebP images
First model and runtime load	About 55 MB	Needed before offline inference

Table 4.6: Frontend Source Modules and Their Responsibilities

Source module	Purpose
Inspect.tsx / Camera.tsx	Upload inspection and supported camera capture
LiveCamera.tsx	Live capture and sequential analysis workflow
captureLifecycle.ts / livePolicy.ts	Resource lifecycle and location or save rules
browserModel.ts / inference.worker.ts	Model initialisation and worker communication
modelCore.ts	Preprocessing, decoding and priority-related inference utilities
InspectionMap.tsx / MapPage.tsx	Map interaction and inspection markers
Report.tsx / localPdf.ts	Report presentation and PDF generation
OfflineWorkspace.tsx / OfflineStatus.tsx	Device records, pending work and offline feedback
storage.ts / inspectionStore.ts	Persistence and inspection storage operations
cloudSync.ts / accountScope.ts	Account-scoped synchronisation
api.ts / browserAPI.ts / transport.ts	Common access to browser or configured server operations

There are two states: no selected image, and image preview, which make up the first half of the inspection page; model loading and model ready make up the second; analysis in progress, successful prediction and failure make up the third. The absence of weights in the prediction list is only considered a success if inference is actually performed; in a failure case and/or in weight absence this is never used as a failback mechanism when the inference

result is lost. Errors are thrown rather than new detections.In case of invalid uploads and unavailable inference, errors are thrown instead of new detections.

4.5 Camera, Location and Map Implementation

Implementation: the behaviour is implemented and turned into software logic, ensuring coverage of software (Section 5.8). Not verified of Moving-Vehicle operation, Real Permission Dialog.

4.5.1 Live Camera Sequence

The Start live camera + GPS control explicitly requests device permissions and prefers the rear camera. The scanner captures a frame, runs sequential inference, displays the analysed snapshot and saves positive sampled frames with the most recent GPS fix. The save policy of Equation (3.19) requires at least one accepted detection and a valid, recent location, so empty frames are not stored automatically and missing or stale GPS prevents saving a frame as a located live inspection. Records retain the original and annotated images, detections, timestamp, priority and session metadata. Predictions initially await human review.

The same physical defect can appear in several saved frames. Unique-defect tracking is not implemented, and neither non-maximum suppression nor retry identifiers merge frames.

4.5.2 Resource Lifecycle

Stop controls and lifecycle handling release the camera and location resources. A permission result that arrives after Stop is released. Scanning is designed to stop when the user leaves or hides the inspection screen, or when a processing or storage error occurs. A frame that is already being committed may finish saving.

4.5.3 Map Behaviour and the Limits of GPS

Leaflet displays online OpenStreetMap tiles [18] with attribution. Users can request their current position or select an inspection point visually. Saved records with coordinates appear as priority-coloured markers. The live capture map follows the device fix and shows the session's capture locations, with blue identifying the device.

GPS reports the device position and not the precise position of a pothole visible ahead, and an image alone cannot establish the geographic coordinates of each defect. Browser

permission, suitable hardware, HTTPS or localhost, and location services are required. Motion blur, viewpoint and sampling gaps can cause missed hazards. The device should be configured before driving and operated by a passenger during travel. The application is an inspection aid and not a driving safety system.

4.6 Web Application, Storage and Reporting

The public website opens without a mandatory sign-in. The primary choices are image upload and camera inspection, manual latitude and longitude entry is not needed, and optional sign-in enables account-backed synchronisation. An uploaded JPEG, PNG or WebP image is validated and previewed, with limits of 10 MB and 25 megapixels. The user can attach a position or proceed without one. The detector returns boxes, classes and confidence scores, the application calculates the priority, and the inspection evidence is saved. A record without a location stays in the history but cannot receive an accurate map marker.

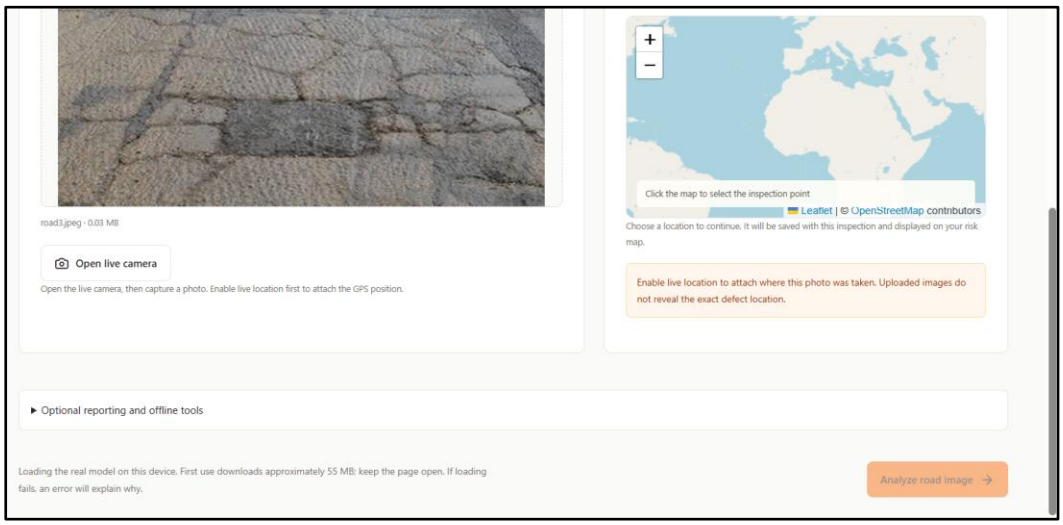


Figure 4.5: Historical Website Screenshot Taken before the Optional-Location and Model-Loader Fixes (Loading State; Not Proof of Current Successful Detection)

Figure 4.5 is a historical user-provided screenshot. It shows a loading and disabled state and is not evidence of current successful detection. The current code removes the earlier upload-location requirement, and dashboard counts are derived from the stored records. Screenshots of the current successful upload, map, camera and history screens still need to be captured from the running application.

Records keep the original and annotated images, model predictions, model settings, timestamps, location where present, notes and review status. Table 4.7 shows where records live in each mode.

Table 4.7: Storage Modes and Their Behaviour

Mode	Storage and behaviour
Guest browser	IndexedDB; records remain on this browser and device
Optional signed-in mode	Account cache; D1 metadata and private R2 evidence synchronisation
Optional Python mode	SQLite records and backend image and report files

Guest history survives an ordinary refresh and reopening unless browser data is cleared or evicted, and it does not transfer automatically to another device. The browser database holds five object stores: records (metadata and image blobs), drafts (photo, location, context and pending-inference state), mutations (queued edits and deletions), settings, and cloudDeletes (deletion operations with revision and conflict details). Guest records use the database RoadGuardOffline, and signed-in accounts use separately named databases, so explicit import is needed to move legacy guest photos into an account. Clearing browser storage removes local evidence, and exporting important records is recommended.

Users can review predictions, reject false positives, edit notes, export records and delete with confirmation. Rejecting a prediction can trigger recalculation of the remaining priority without hiding the original inference evidence. Table 4.8 lists the workflow states, which are not independent proof that repair work occurred.

Table 4.8: Review Status Labels

Status	Meaning
awaiting_verification	Model output has not been confirmed
verified_damage	A reviewer marked damage as verified
false_alarm	The inspection was marked as a false alarm
maintenance_completed	The record was marked as completed maintenance

A manifest and service worker support progressive-web-app operation. The app assets, the model and the local WebAssembly runtime must have downloaded successfully before

offline use, and the first load is about 55 MB. Installing the PWA alone does not supply uncached inference resources. Pending records persist locally and can be processed or synchronised when the required runtime or server is available. Uncached online map areas require internet, while saved locations remain available in record and coordinate views without tiles. Optional controls retain up to nine nearby tiles from an installed, legally usable local tile package. No package or entire-city map is bundled, and no automatic bulk download of OpenStreetMap is performed.

Local PDF code produces inspection reports with original and annotated images, predictions, timestamps, location, priority and model information, and CSV and JSON exports support record portability. Authority reporting is an in-app queue only. No government API or authority email delivery is configured, so external delivery is not claimed.

On privacy, guest on-device inference does not send photos to an external detection service. Account synchronisation transfers evidence to the configured hosted service and Python mode transfers data to the configured backend, so the user must know which mode is selected. Map tile requests disclose the viewed area to the tile provider but do not contain the inspection photo or notes.

4.7 Optional Backend, Software Environment and Reproducibility

The optional Python backend exposes health, model-information, analysis and inspection operations through FastAPI. Ultralytics loads the checkpoint once at startup and SQLite initialises local persistence, with MODEL_PATH and MODEL_VERSION configuring the model. SQLite stores each inspection as a JSON payload in an analyses table, beside deleted_records tombstones, record_operations for edit-retry deduplication and review_events. Images and generated reports are files linked by the record. The hosted account service uses separate D1 and R2 infrastructure and must not be described as this local database. The public browser app does not reach a Python process on a laptop automatically, so remote Python mode needs a reachable HTTPS host and a matching CORS configuration. Table 4.9 lists the principal routes.

Table 4.9: Principal Routes of the Optional Python Backend

Method and path	Purpose
GET /api/health	Service and model availability
GET /api/model/info	Model identity, classes and settings
POST /api/inspections/analyze	Analyse and save an image (aliases exist)
GET /api/inspections	Paginated search and filter
GET /api/inspections/nearby	Distance-based query of located records
GET /api/dashboard	Aggregated inspection statistics
PATCH /api/inspections/{id}/review	Revision-checked review update
PATCH /api/inspections/{id}/edit	Details and rejected-prediction edit
DELETE /api/inspections/{id}	Delete with an operation ID
GET /api/inspections/{id}/report.pdf	Generate or retrieve the PDF report
GET /api/authorities/queue	In-app queue; no external delivery

Validation follows the rules of Section 3.7. Report details constrain the length of address, place, notes and authority fields, an optional Indian pincode must be six digits with a nonzero first digit, and empty fields stay allowed where defined and are never replaced with invented addresses. The backend answers invalid report details, client identifiers or locations with 422, a client identifier reused with different data with 409, replay of a deleted identifier with 410, an unavailable trained model with 503, and an analysis exception with 500 and a generic user message. If PDF generation fails after a saved analysis, the inspection is retained and the report status records the failure. Protected modification routes use the configured reviewer credential in Python mode, which must be kept private.

For the frontend, install Node 24 and pnpm 11.25.0 (npm install -g pnpm@11.25.0), then run the following commands from the project root. Production service-worker caching is enabled in production builds and not in ordinary source development, and the included browser model does not require a Python server.

pnpm install

pnpm dev

pnpm build

For the optional backend on Windows, enter the backend folder and run the commands below, keeping the generated reviewer credential private.

```
py -3.11 -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
Copy-Item .env.example .env
.venv\Scripts\python.exe configure-reviewer.py
.venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8000
```

The environment settings are MODEL_PATH (default ./models/best.pt), MODEL_VERSION (road-hazard-yolov8s-640-3bc31858), MODEL_IMAGE_SIZE (640) and CONFIDENCE_THRESHOLD (0.25). The health page and API documentation are then available locally on port 8000.

The checkpoint identity, as a SHA-256 hash, is:

3bc3185868a8c8f6e65b9f46af4eb65e2da62e62a213dfc4f4e46646a26f1208

To replace the model, install an actual trained checkpoint, verify its class order and input and output shapes, export a compatible ONNX graph, update the release hashes and version, and repeat the numerical parity and real-image checks before publishing. Metrics of the old model must not be kept as if they described the new release. Table 4.10 lists the saved artefacts from which this report's numbers and figures are taken.

Table 4.10: Evidence Used in This Report

Evidence	Purpose
dataset_audit.json	Retained counts, instances and split protocol
model_metadata.json	Architecture, training, validation and latency
test_metrics.json	Held-out overall and class-wise results
results.csv	91-epoch loss and validation learning curves
rerun_predictions.json	Actual fresh ONNX predictions
China_Drone__China_Drone_000022.jpg	Clean input for the prediction comparison
image(8).png	Historical user-provided website screenshot
README.md and VALIDATION.md	Deployment instructions and recorded test scope
src/livePolicy.ts and risk.py	Current GPS-save rule and priority formula

Keep the trained checkpoint, its metadata, the evaluation JSON and the original notebook outside transient runtime storage, together with the model hash. Export local inspection records before clearing site data. A notebook renamed to .pt is not a model checkpoint.

CHAPTER 5

RESULTS, BENCHMARKS, AND DISCUSSION

This chapter gives the results of what was measured, and what the results enable us to conclude. All graphics and numbers are from a saved set of artefacts (see 4.10). No adjustments or additions are made for this report and where an item has not been measured there will be no change, the report will state that. This section reports on the set-up of the experiment and the evaluation protocol. The 24,976 images in the trained dataset were used as training images while the 5,339 images in the validation set were used for monitoring the recorded performance of the deployed YOLOv8s model (Table 4.2). It was tested using the 5,175 test images that have not been used in the training process. A set of separate experiments, that were recorded in a YOLO11s development notebook, with a different preparation of samples containing 5,758 images for development evaluation are reported separately in Section 5.6. Below, only the Tesla T4 hardware has been documented as it was the single machine used as the latency machine. The chapter is organised by 4 evaluation questions.

E1. Learning behaviour: training and validation curves are converging, what is the validation behaviour and is it similar to training loss?

E2. Held out detection quality: the precision (and recall and mean average precision) of the deployed checkpoint on the held out split.

E3. Class-wise behaviour – which classes are it working well on and where is it falling behind? The results are discussed with the class counts in Table 4.2 but it is not claimed that there is causation.

E4. Software and workflow checks: What checks have been completed/not completed, what behaviours are recorded or not documented?

The definitions of the metrics as stated in Section 3.5 are followed. The data on the test scores do not constitute the validation learning curves.

5.2 Training Behaviour from Saved Logs

Figure 5.1 and Figure 5.2 plot the saved 91-epoch results.csv. They show actual logged values and not illustrative curves.

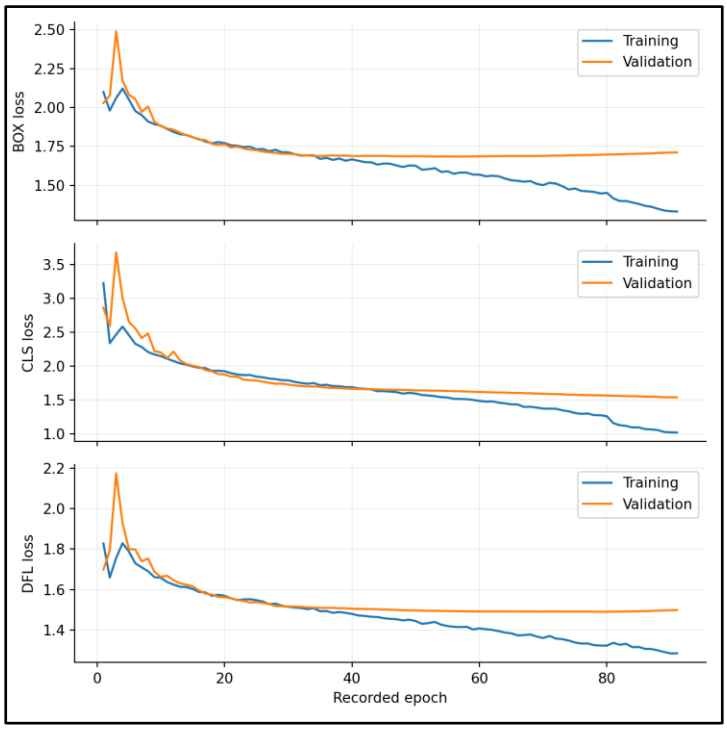


Figure 5.1: Recorded Training and Validation Box, Classification and Distribution Focal Losses

Training losses decrease over the run. Validation box and distribution focal losses flatten while training loss continues to improve, which cautions against selecting a checkpoint solely from training loss and agrees with the use of validation fitness for checkpoint selection. All three losses show a short rise in the first few logged epochs before they settle.

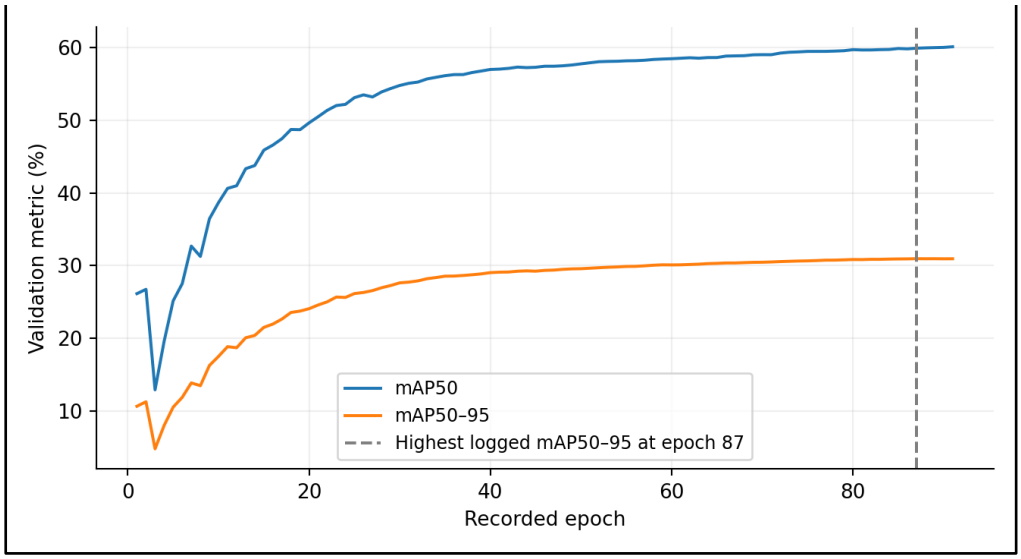


Figure 5.2: Validation mAP across Logged Epochs; the Dashed Line Marks the Highest Logged mAP50–95 (Epoch 87)

Validation mAP50 rises steeply over the early epochs and then flattens, and the highest logged mAP50–95 occurs at epoch 87 of the 91 logged. This answers E1: the curves show convergence, with little gain in the final epochs. The requested 80 epochs and the 91 logged epochs differ, and the recorded configuration notes that the run used a time budget.

5.3 Held-Out Test Performance

Table 5.1 gives the held-out test metrics of the deployed checkpoint, read from test_metrics.json.

Table 5.1: Held-Out Test Metrics of the Deployed YOLOv8s Checkpoint

Metric	Value	Reading
Precision	67.71%	Share of predictions that matched a reference box
Recall	52.83%	Share of reference boxes that were found
mAP50	59.02%	Mean average precision at IoU 0.50
mAP50–95	30.51%	Mean average precision over IoU 0.50 to 0.95

mAP50 measures detection average precision at IoU 0.50 and mAP50–95 averages over stricter overlap thresholds. Neither is classification accuracy. A recall of 52.83% means that about 47% of the labelled instances were not found at the evaluation settings of the saved test run. These values answer E2.

5.3.1 Runtime Measurement

A saved Tesla T4 benchmark used 20 samples after 5 warm-up iterations. It recorded a mean processing time of 15.50 ms, a median of 15.71 ms and a 95th percentile of 19.17 ms. The measurement included image reading, preprocessing, prediction and NMS, and excluded drawing and saving. It is not a measured phone or browser live-camera frame rate.

5.4 Class-Wise Performance

Table 5.2 reports the class-wise held-out results, and Figure 5.3 compares AP50 with recall.

Table 5.2: Class-Wise Held-Out Results of the Deployed Checkpoint

Class	Precision	Recall	AP50	AP50–95
D00	65.67%	53.26%	58.74%	32.91%
D10	66.58%	52.80%	59.52%	29.99%
D20	70.71%	62.09%	68.25%	36.61%
D40	67.88%	43.16%	49.57%	22.55%

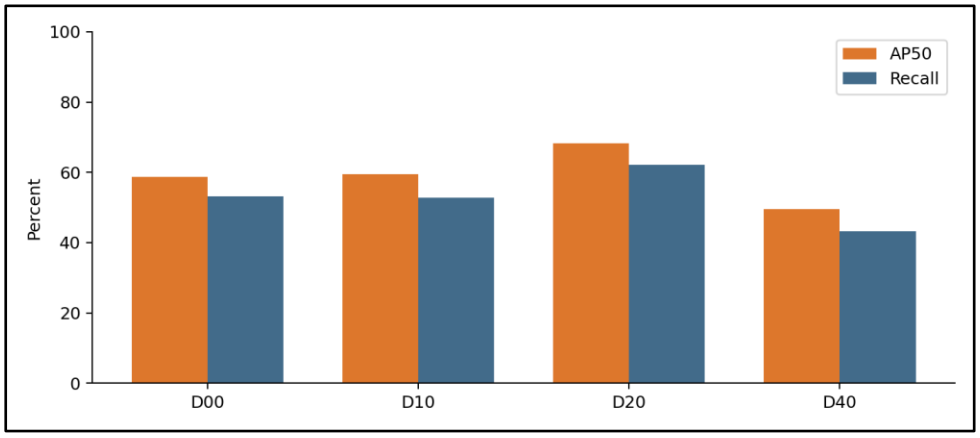


Figure 5.3: Class-Wise Held-Out AP50 and Recall from test_metrics.json

D20 alligator cracks have the strongest AP50 (68.25%) and the highest recall (62.09%). D40 potholes have the weakest AP50 (49.57%) and the lowest recall (43.16%), so missed potholes are a significant limitation. D40 also has the fewest instances in the prepared data (Table 4.2). The two facts are consistent with each other, but this study did not test whether one causes the other. These findings answer E3.

5.5 Precision–Recall Behaviour and Confidence Threshold

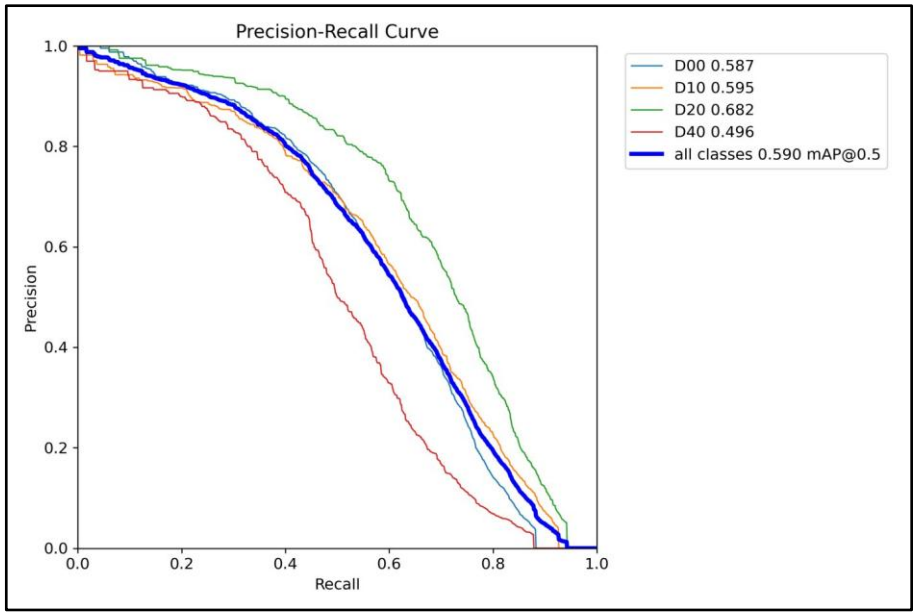


Figure 5.4: Saved Deployed YOLOv8s Held-Out Precision–Recall Plot; All-Class AP50 Is Approximately 0.590

The class curves in Figure 5.4 are consistent with Table 5.2: D20 has the highest AP50 and D40 is lower. Precision generally decreases as the operating point admits more predictions and recall rises. The appropriate operating point depends on the cost of missed defects against the review workload created by false alarms.

The effect of the confidence threshold is quantified in the YOLO11s notebook, which evaluated its 512-pixel checkpoint on 5,758 validation images. The saved matching protocol is same class, IoU at least 0.50 and one-to-one matching. This experiment belongs to the notebook model and not to the deployed YOLOv8s model, for which no threshold sweep is recorded. Figure 5.5 and Table 5.3 show the result.

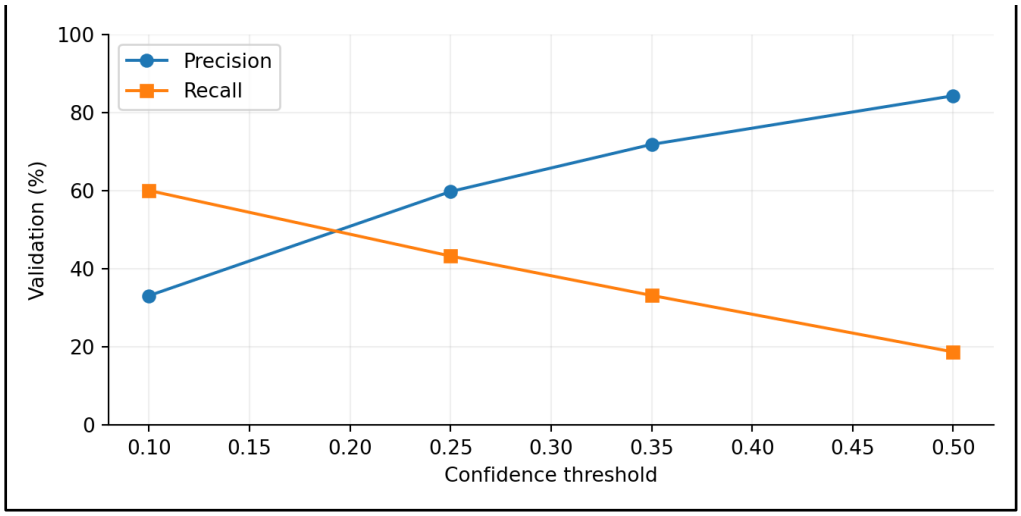


Figure 5.5: Notebook Validation Precision–Recall Trade-off across Confidence Thresholds (YOLO11s, 512 Pixels)

Table 5.3: Confidence Threshold Experiment on the YOLO11s 512-Pixel Notebook Checkpoint

Threshold	TP	FP	FN	Precision	Recall
0.10	4,911	9,911	3,266	33.13%	60.06%
0.25	3,539	2,380	4,638	59.79%	43.28%
0.35	2,715	1,063	5,462	71.86%	33.20%
0.50	1,532	285	6,645	84.31%	18.74%

Raising the threshold reduced false positives but also discarded genuine damage predictions. The 84.31% precision at 0.50 is not overall accuracy and has to be read together with the recall of 18.74%. Raising a threshold is not the same as training a more accurate pothole model. A saved confusion-matrix image also exists, but its background counts should not be mixed with threshold-specific precision and recall without confirming the generation settings, so the numerical tables and precision–recall plots are the evidence used here.

5.6 Comparison with the YOLO11s Notebook Experiments

5.6.1 Overall Notebook Results

The supplied final-project.ipynb provides a second, distinct experimental record. Its stored outputs document YOLO11s candidates evaluated at 512 and 640 pixels. These results are genuine saved notebook outputs, but the exact checkpoint bytes were not recovered for website deployment. Table 5.4 gives the overall results.

Table 5.4: Notebook YOLO11s Test Results (Separate from the Deployed Model)

Notebook candidate	Precision	Recall	mAP50	mAP50–95
YOLO11s 512	55.18%	46.98%	48.07%	23.02%
YOLO11s 640	55.75%	46.69%	48.67%	23.44%

Both notebook test evaluations report 5,758 images and 8,088 labelled instances. The deployed YOLOv8s evaluation used 5,175 held-out images from a different preparation (Table 5.1). A numerical difference between the notebook and the deployed model therefore cannot be attributed solely to the architecture, and the two sets of results are kept in separate tables and not merged into one best-result table.

5.6.2 Class-Wise Notebook Results at 640 Pixels

Table 5.5 gives the class-wise notebook results at 640 pixels, as rounded in notebook cell 51.

Table 5.5: Class-Wise Notebook Results at 640 Pixels

Class	Precision	Recall	AP50	AP50–95
D00	58.3%	49.6%	51.6%	26.9%
D10	51.6%	48.2%	46.8%	21.3%
D20	62.6%	54.3%	59.0%	29.5%
D40	50.5%	34.7%	37.3%	16.0%

The 640-pixel candidate slightly increased overall mAP relative to the 512-pixel candidate, but its reported recall was slightly lower, so the comparison is not evidence that every damage category improved. As for the deployed model, D20 has the highest AP50 and D40 the lowest. The availability of verified checkpoint bytes and exportable inference decided which model was packaged. The recovered YOLOv8s checkpoint was deployed and its output parity checked, while the notebook experiments remain part of the research history.

5.7 Prediction and Priority Worked Examples

The clean China Drone image is 512 × 512 pixels. A fresh execution of the actual ONNX model on it, carried out on 14 October 2026, is shown in Figure 5.6. The predictions are not manually confirmed labels.

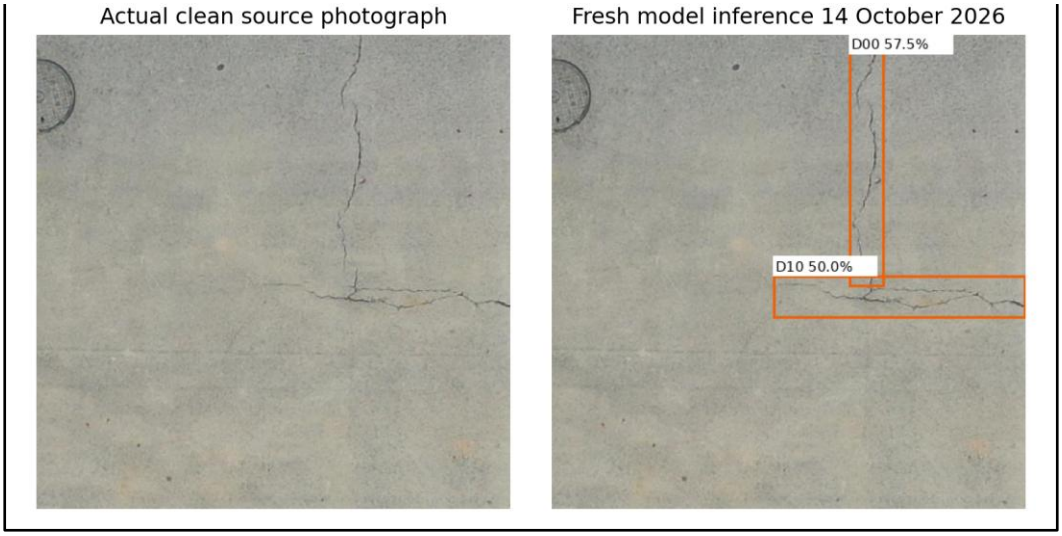


Figure 5.6: Actual Clean Road Photograph and Fresh ONNX Inference Executed on 14 October 2026

Table 5.6: ONNX Predictions for the Clean 512 × 512 Image

Prediction	Confidence	Box coordinates x1, y1, x2, y2
D00	57.4869%	321.91, 0.00, 359.15, 272.11
D10	49.9738%	239.82, 260.62, 512.00, 306.37

The score of Equation (3.13) uses B = 20, because D10 is the highest baseline present, N = 2, K = 3 and the default context C = 0. The summed normalised box coverage is approximately A = 0.08616. Therefore S = round(20 + 20 × 0.08616 + 3) = 25, which is Low priority. This is a transparent calculation from predicted boxes and not a measured engineering severity.

Table 5.7 lists three further actual runtime checks. The India and Japan images are parity source images that already carry annotations. They suit runtime consistency checks and not clean-input accuracy claims.

Table 5.7: Additional Runtime Checks and Rule Outputs

Test input	Accepted predictions	Rule output
test-road.jpg	None	0, Low; no road-safety certification
India__India_000027.jpg	D40, confidence 56.70%	45, Moderate
Japan__Japan_000028.jpg	D00, confidence 51.00%	15, Low

The clean-image inference took approximately 1,242.87 ms in ONNX Runtime Web WASM, single-threaded under Node. This is not a browser camera latency benchmark, and it is not directly comparable with the Tesla T4 timing of Section 5.3 without accounting for runtime and hardware differences. A reviewer should inspect whether each box encloses the intended damage, whether the class is plausible and whether other defects were missed. Confidence and priority are preserved separately, and a rejection removes that prediction from later screening calculations without concealing the original inference evidence.

5.8 Software Validation and Verification Status

Table 5.8 lists the outcomes recorded in the project validation documentation. They are prior test results, and this report does not claim that a fresh full test suite was executed while it was being prepared.

Table 5.8: Recorded Software Test Outcomes

Test area	Recorded result
Checkpoint and ONNX graph	Checksum matched; graph checker passed
PyTorch / ONNX WASM parity	Four real images; confidence difference below 0.001; box difference below 0.2 px
Backend regression	20 tests passed
Actual backend workflow	Upload, inference, annotation, risk, SQLite reopen and PDF passed
Storage adapters	Save and reopen, ID deduplication, history, review, conflict and deletion passed
Account synchronisation	Ownership, retry and conflict logic covered by automated tests
Service worker	Routing and cache tests passed in a sandbox
Live policy	Capture-time GPS and positive-frame persistence software checks recorded
Build	TypeScript and production build passed

Some adapter tests substitute recorded real inference at the model boundary to isolate storage behaviour, and they are not new model-accuracy experiments. Service-worker sandbox tests are not installed-browser tests. Table 5.9 summarises, feature by feature, what is implemented, what has been checked and what remains open. This answers E4.

Table 5.9: Implementation and Verification Status of the Project Features

Feature	Status	Evidence or remaining work
Four-class detection with boxes and confidence	Implemented and checked	Held-out evaluation (Table 5.1, Table 5.2)
In-browser ONNX inference	Implemented and checked	Parity on four real images; checksum matched
Upload inspection without location	Implemented and checked	Storage and backend workflow checks; hosted end-to-end interaction needs physical acceptance
Screening priority rule	Implemented; unvalidated	Not validated against engineer-assigned severity
Live camera and GPS saving	Implemented; not physically verified	Software policy checks only; moving-vehicle use and real permission dialogs unverified
Map markers	Implemented; not physically verified	Current successful screenshot still needed
Offline PWA operation	Implemented; not physically verified	Sandbox tests only; installed close-and-reopen test needed
Account synchronisation	Implemented; partly checked	Automated tests; sign-in on devices unverified
Browser PDF report	Implemented; not physically verified	Python-workflow PDF generated; browser layout needs acceptance
Authority reporting	In-app queue only	No external delivery configured
Unique-defect tracking across frames	Not implemented	Repeated frames can duplicate one defect
Bundled offline map tiles	Not implemented	Optional local package of up to nine nearby tiles only

Physical acceptance still needs to be done. The remaining work can be broken down into the following 10 steps. This does not imply that they have passed, but the inclusion only detracts from the process used to teach them.

1.A road test should take place with a passenger or in a controlled stationary environment, not by a driver, operating the interface. Get access to the public URL, without logging in, and wait for the Model ready condition.

2. Upload a known positive (with NO location), confirm boxes, scores & saved history.

3. Take any GPS fix (or take a map point manually) and check out the save marker. No, upload a safe valid image that has no prediction in the list of accepted predictions.

4.Confirm that safety isn't the winning prediction.

5. Permit camera/deny GPS, then repeat if allowed; Ensure that errors and no false GPS coordinates are devised.

6. Operate a controlled camera session, review positive-frame records and terminate and review resources (check releases).

7. Open and save an inspection and export PDF, CSV and JSON; compare actual values and evidence. Disconnect the internet and open up the APP, Check the local history and Map fallback.

8. Successfully cache asset, make sure to do offline during the purchase of the game.

9. Run 2 synchronisations; test 1 on a magnitude of changes, ID synced, and explicit ID revision conflicts.

10. Review design on desktop and on a phone (on a narrow screen).

5.9 Discussion

The results, within the limits of the data obtained, indicate an improvement in the ability of this detector to identify approximately half of the damage instances and that this detector is more reliable at alligator crack damage sites than it is at pothole damage sites. The model deployed achieves a mAP50 label held-out split of 59.02% and a mAP50–95 label held-out split of 30.51%. The class of highest baseline in the screening rule (Section 3.6) is also the class that is least accurately recalled (43.16%), showing that treatment of the highest class in the screening rule is a class that the detector will surface on the least frequently. This was the most significant deficiency that was found in the project. The threshold experiment demonstrates the need to have more than one typical number. Even at the lowest level in Table 5.3, the notebook model succeeds in labelling 60.06% of the instances, while it is mainly missing its predictions at the highest level, 84.31% of the instances are correctly identified by its prediction. The operating point needs to be determined based on the cost of missed defects as well as review workload and should not be reported as an improvement to the model. The behaviour of the screening priority is as expected for the examples in Section 5.7. It is transparent (all terms can be linked to a box, a class or a context input). Additionally, it is not validated, so its output orders need records for human evaluation – it conveys no information on risk issues. The three software programs considered in Table 5.8 include some useful features. Section 5.8 still requires to be completed on the physical manikin for all the camera, GPS, offline, account and browser-PDF behaviours. The results are not representative of road performance on all Indian roads, during night time, on rainy days, on moving vehicles or images of such vehicles, as the test split had not yet been annotated by the officials and is not a route-disjoint split. The correct answer is that this deployed model realized 59.02% mAP50 and 30.51% mAP50-95 on its labeled held-out split; and its success rate of 43.16% pothole recall suggests that there is still a significant shortcoming: potholes that were not detected. Inaccuracy or field-test scores are not added for invented accuracy.

CHAPTER 6
CONCLUSION AND FUTURE SCOPE

6.1 Summary of Completed Contributions

The project puts into use an operational inspection application based on a practical 4 class road damage detector. It detects the following crack types: Longitudinal Cracks, Transverse Cracks, Alligator Cracks, Potholes; displays boxes and confidence scores; calculates a documented screening priority, saves evidence for review with a review status and displays found evidence on a map. The exported detector runs within the browser itself, meaning that there is no need for a separately hosted Python server (default workflow) and local records are supported with optional synchronisation, offline resources, PDF, CSV and JSON export. The following are the results measured. On 5,175 held-out images the deployed YOLOv8s checkpoint reached precision 67.71%, recall 52.83%, mAP50 59.02% and mAP50–95 30.51%. D20 had the strongest AP50 at 68.25%, and D40 the weakest at 49.57% with a recall of 43.16%. The mean was 15.50ms/frame for each image for a saved Tesla T4 benchmark. The software that was recorded for example includes a matching checkpoint checksum, parity within 0.001 of the PyTorch confidence, and 0.2 of the box position on four real images, 20 passing back end tests and a production build. It's not a new detector architecture per se that's defined by the contribution, it's integration and transparent management of inspection.

6.2 Key Theoretical and Practical Takeaways

There are three noteworthy lessons. The first thing is that the failure states of the application should not be lying: If the inference has been made, then the prediction list will not indicate a failure otherwise, it will describe it as a failure to save; and predictions should not be counted as persisted even if they failed to save, nor should they be counted as pending until a person confirms them. Second, metrics must be interpreted as their being measures of something. The terms accuracy, precision, value and threshold are meaningless without the corresponding terms in recall, and without the operating point (respectively without the improvement of the model). Thirdly, the location / evaluation statements must be explicitly

limited. Finally, GPS reports on the device, non-maximum suppression does not merge repeated frames and without route identifier a split can't exclude similar scenes in training data and test data.

6.3 Limitations

The limits are given to enable reading of the results correctly. The rate of pothole recall scores at 43.16% is still low. The field data used in the test are not independent, but route-disjoint field data. There is no check to validate against engineer assigned severity, accident outcome or repair cost or maintenance schedules. The GPS is operating at the device level and repeated-on-the-fly blocks may result in multiple physical defects, and an external authority delivery isn't set up. Depth of pothole, width of crack and capacity of structure are not calculated by the model. Unverified on real devices are: moving-vehicle performance, real camera permission dialog, real GPS permission dialog and layout of installed-PWA for offline reopening in PDF file format on the device (browser) itself. There are no screenshots of current successful websites and the only website screenshot in this report is a historical one.

6.4 Future Research Directions

The following are the limits that specific to future work needs to be accomplished and in the order listed.

1. Gather impartial road images individually from local roads, labeled with road type at route level and validated by specialists, with separation between road types.

2.The to-do list includes extending the pothole examples in the train set as well as determining the class imbalance of the train and test set without tuning on the test set.

3. Make special evaluations for cases of night, rain, and shadow, and for motion-blur cases.

4. Confirm priority vs Engineer assigned Severity and operational maintenance choices.

5. Figure out the physical size or exact position of the defect, before adding geometry and depth (or calibrated sensing).

6. Explore the options of temporal tracking and spatial grouping in order to decrease the number of live records.

7. Fill in the sections "physical-browser", "offline", “camera”, and “gps” of the list of items to be tested for in Section 5.8 for mobile acceptance testing.

Integrate an authorised authority endpoint only once you have agreed on and established consent, delivery acknowledgement and data governance.

REFERENCES

[1] Koch, C., Georgieva, K., Kasireddy, V., Akinci, B., & Fieguth, P. (2015). A review on computer vision based defect detection and condition assessment of concrete and asphalt civil infrastructure. Advanced Engineering Informatics, 29(2), 196–210.

[2] Mednis, A., Strazdins, G., Zviedris, R., Kanonirs, G., & Selavo, L. (2011). Real time pothole detection using Android smartphones with accelerometers. In Proceedings of the 2nd International Workshop on Mobility in Wireless Sensor Networks (MobiSensor), Barcelona, Spain, 1–6.

[3] Maeda, H., Sekimoto, Y., Seto, T., Kashiyama, T., & Omata, H. (2018). Road damage detection using deep neural networks with images captured through a smartphone. arXiv:1801.09454.

[4] Arya, D., Maeda, H., Ghosh, S. K., Toshniwal, D., Mraz, A., Kashiyama, T., & Sekimoto, Y. (2020). Transfer learning-based road damage detection for multiple countries. arXiv:2008.13101.

[5] Arya, D., Maeda, H., Ghosh, S. K., Toshniwal, D., & Sekimoto, Y. (2022). RDD2022: A multi-national image dataset for automatic road damage detection. arXiv:2209.08538 (later published in Geoscience Data Journal, 2024).

[6] Arya, D., Maeda, H., Ghosh, S. K., Toshniwal, D., Omata, H., Kashiyama, T., & Sekimoto, Y. (2022). Crowdsensing-based road damage detection challenge (CRDDC-2022). arXiv:2211.11362. Big Data Cup, IEEE International Conference on Big Data 2022.

[7] ASTM International. Standard practice for roads and parking lots pavement condition index surveys (ASTM D6433). West Conshohocken, PA, USA.

[8] ONNX Runtime developers. (2021). ONNX Runtime [Computer software]. https://onnxruntime.ai

[9] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779–788.

[10] Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards real-time object detection with region proposal networks. In Advances in Neural Information Processing Systems 28 (NeurIPS).

[11] Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., & Berg, A. C. (2016). SSD: Single shot multibox detector. In European Conference on Computer Vision (ECCV), 21–37.

[12] Wang, C.-Y., Bochkovskiy, A., & Liao, H.-Y. M. (2023). YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 7464–7475.

[13] Wang, C.-Y., Liao, H.-Y. M., Wu, Y.-H., Chen, P.-Y., Hsieh, J.-W., & Yeh, I.-H. (2020). CSPNet: A new backbone that can enhance learning capability of CNN. In IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), 390–391.

[14] Zheng, Z., Wang, P., Liu, W., Li, J., Ye, R., & Ren, D. (2020). Distance-IoU loss: Faster and better learning for bounding box regression. In Proceedings of the AAAI Conference on Artificial Intelligence, 34(7), 12993–13000.

[15] Li, X., Wang, W., Wu, L., Chen, S., Hu, X., Li, J., Tang, J., & Yang, J. (2020). Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection. In Advances in Neural Information Processing Systems 33 (NeurIPS), 21002–21012.

[16] Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8 [Computer software]. https://github.com/ultralytics/ultralytics

[17] Khanam, R., & Hussain, M. (2024). YOLOv11: An overview of the key architectural enhancements. arXiv:2410.17725.

[18] OpenStreetMap contributors. OpenStreetMap. https://www.openstreetmap.org (map data available under the Open Database License).

[19] National Institute of Standards and Technology. (2015). Secure Hash Standard (SHS). Federal Information Processing Standards Publication 180-4.

[20] Zauner, C. (2010). Implementation and benchmarking of perceptual image hash functions (Master's thesis). Upper Austria University of Applied Sciences, Hagenberg, Austria.

[21] Rezatofighi, H., Tsoi, N., Gwak, J., Sadeghian, A., Reid, I., & Savarese, S. (2019). Generalized intersection over union: A metric and a loss for bounding box regression. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 658–666.

[22] Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. In European Conference on Computer Vision (ECCV), 740–755.

[23] Neubeck, A., & Van Gool, L. (2006). Efficient non-maximum suppression. In 18th International Conference on Pattern Recognition (ICPR), 3, 850–855.